

Mechanics of Composite Materials

Project 2

Luncheng Zhang

Kiefer Gonsalves

Word Count: 347 words

Introduction:

This report investigates the elastic and thermal behavior of composite laminates using two different materials, AS4/3501-6 (carbon/epoxy) and E-glass/epoxy, under varying fiber orientations and stacking sequences. The goal is to evaluate the normalized transformed elastic properties of individual laminate, compute the effective laminate moduli using Classical Laminate Theory (CLT), and estimate thermal residual stresses after curing.

Methods:

For Part 1, normalized transformed properties— E_2'/E_1 , G_{12}'/G_{12} , and ν_{12}' —were calculated for fiber orientations ranging from 0° to 90° in 10° increments. The Q matrix was computed using material properties and transformed using θ . For laminate configurations, the A matrix was assembled from $\bar{Q}$ matrices and used to extract effective moduli (E_x , E_y , G_{xy}) for three stacking sequences: $[0/90]_s$, $[0_2/90_2]$, $[0/90/\pm 45]_s$ (Quasi-isotropic). Additionally, $[\pm\theta]_s$ symmetric angle-ply laminates were analyzed for trends in stiffness versus angle.

For Part 2, thermal residual stresses σ_1 and σ_2 were calculated for $[0/90]_s$ and $[0_2/90_2]$ layups using coefficients of thermal expansion (CTEs) and laminate stiffness. Curing occurred at 177°C with cooling to room temperature (20°C). Material strengths were compared to stress results. The stresses were derived using classical laminate theory-based expressions that relate thermal expansion and modulus mismatch through equilibrium equations. All calculations were implemented using Python. Functions were created to compute the Q matrix based on engineering constants, apply transformation equations for different fiber orientations, and construct the A matrix for laminate-level analysis. Python's NumPy and Matplotlib libraries were used for computation and visualization.

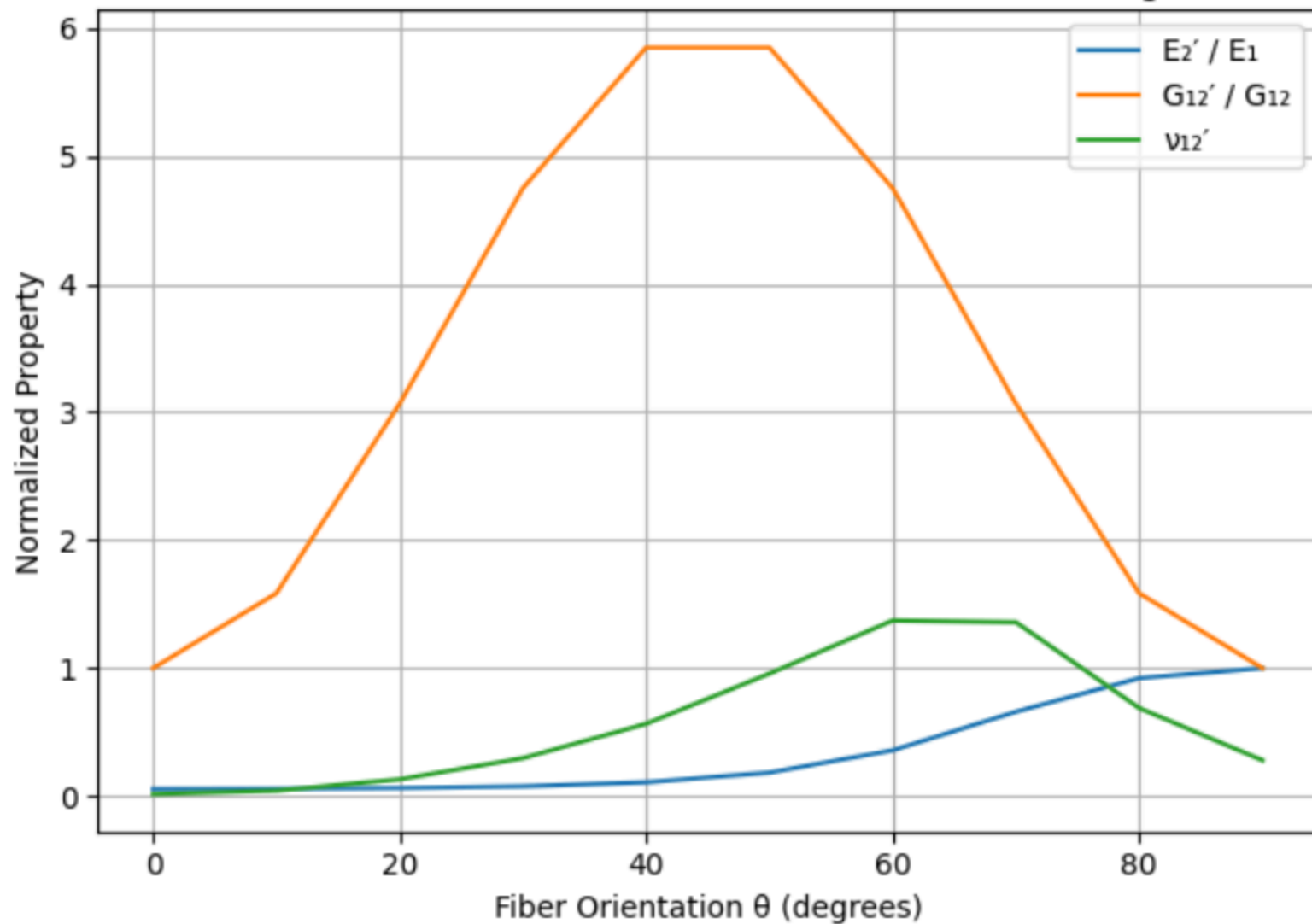
Results:

Part One:

==== AS4/3501-6 ====

Angle (deg)	E2'/E1	G12'/G12	ν_{12}'
0	0.057	1.000	0.016
10	0.058	1.585	0.044
20	0.064	3.065	0.132
30	0.077	4.749	0.297
40	0.109	5.847	0.565
50	0.184	5.847	0.957
60	0.358	4.749	1.373
70	0.660	3.065	1.359
80	0.922	1.585	0.689
90	1.000	1.000	0.280

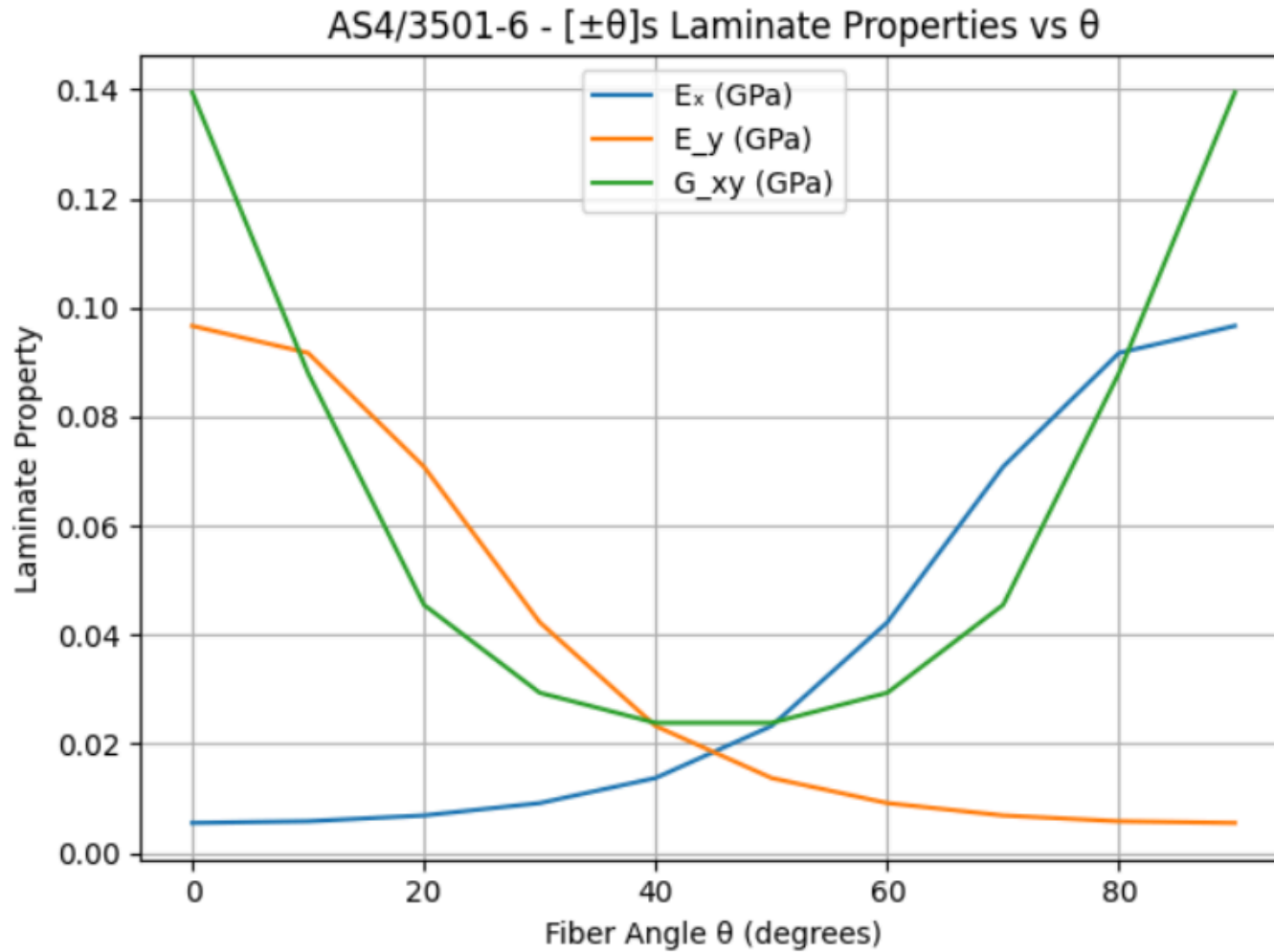
AS4/3501-6 - Normalized Lamina Moduli vs Fiber Angle



$[0/90]_s$: $E_x = 0.01$, $E_y = 0.01$, $G_{xy} = 0.14$ (GPa)

$[02/902]_s$: $E_x = 0.01$, $E_y = 0.01$, $G_{xy} = 0.14$ (GPa)

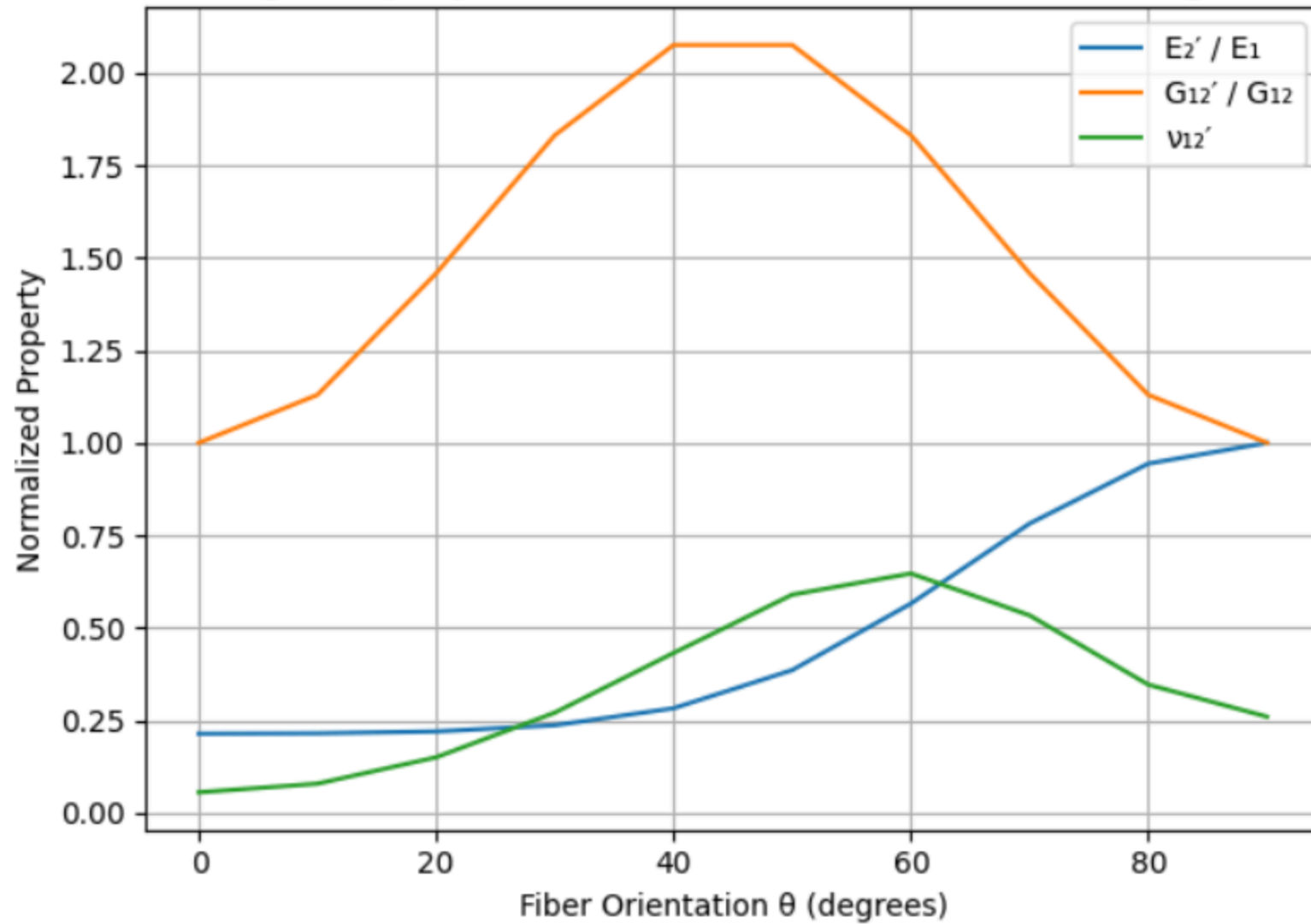
$[0_m/90_n/\pm 45_p]_s$: $E_x = 0.01$, $E_y = 0.01$, $G_{xy} = 0.04$ (GPa)



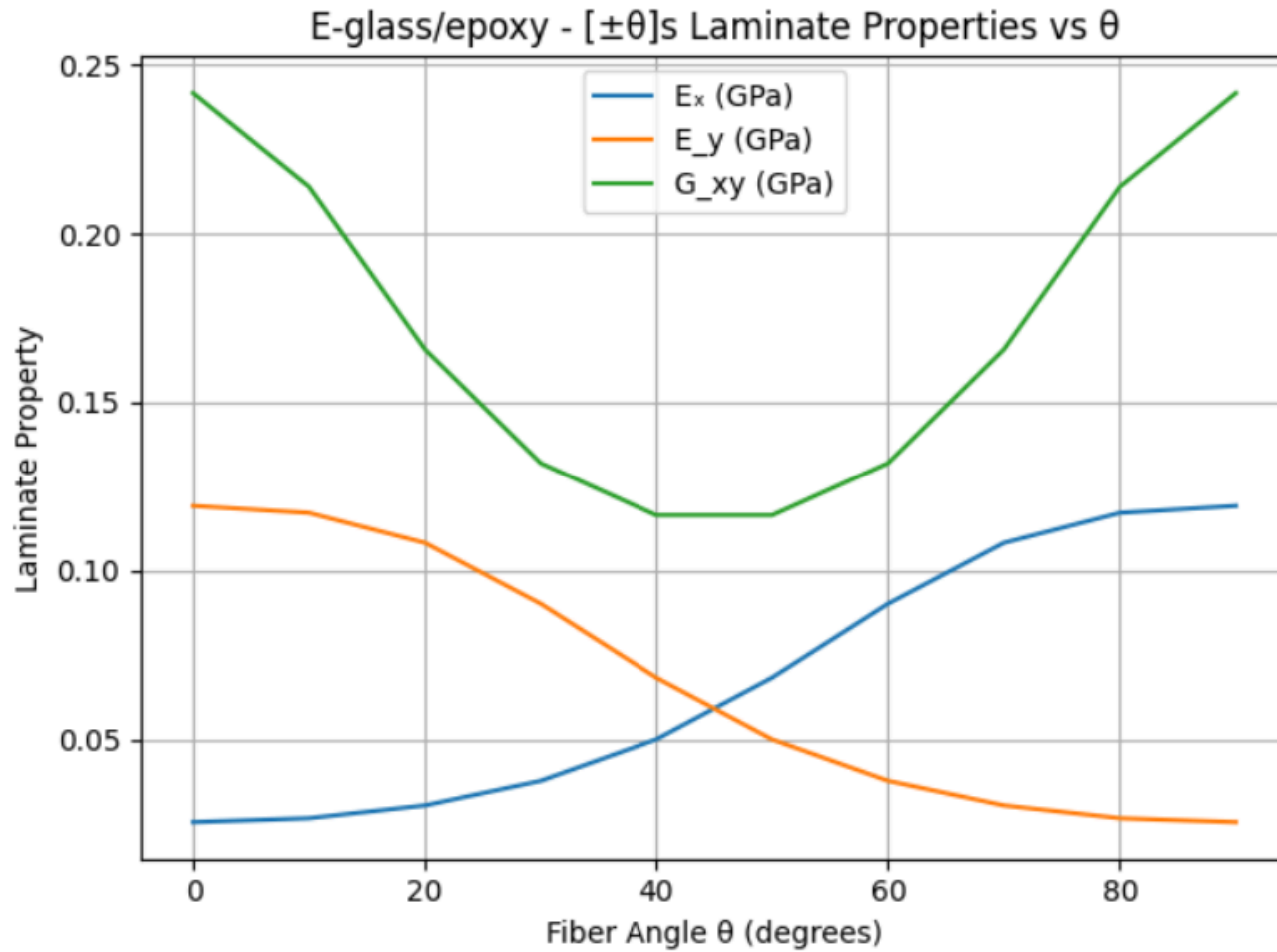
==== E-glass/epoxy ====

Angle (deg)	$E2'/E1$	$G12'/G12$	$\nu12'$
0	0.214	1.000	0.056
10	0.215	1.130	0.079
20	0.220	1.458	0.150
30	0.237	1.831	0.271
40	0.283	2.075	0.432
50	0.386	2.075	0.590
60	0.566	1.831	0.647
70	0.782	1.458	0.534
80	0.944	1.130	0.347
90	1.000	1.000	0.260

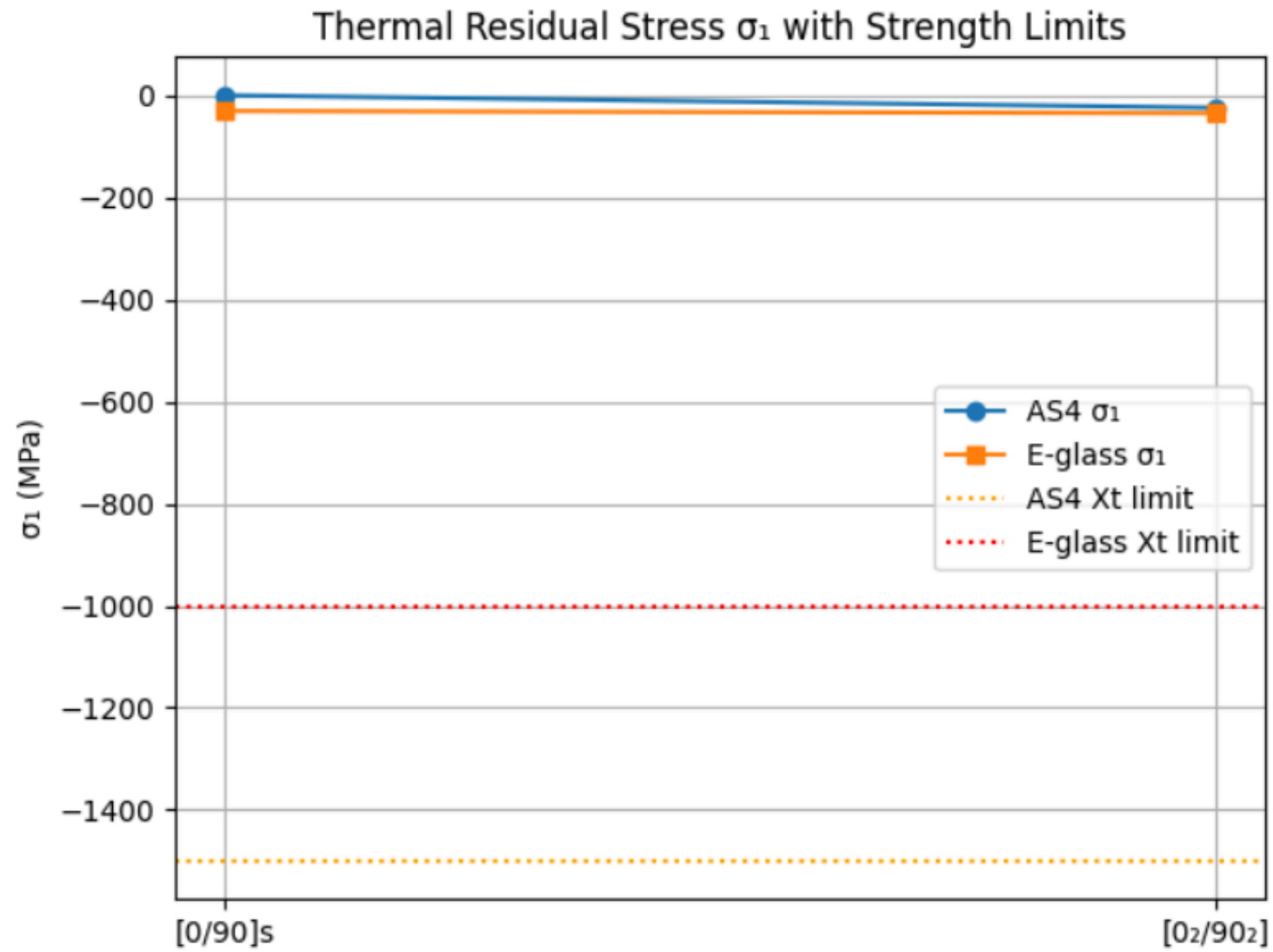
E-glass/epoxy - Normalized Lamina Moduli vs Fiber Angle



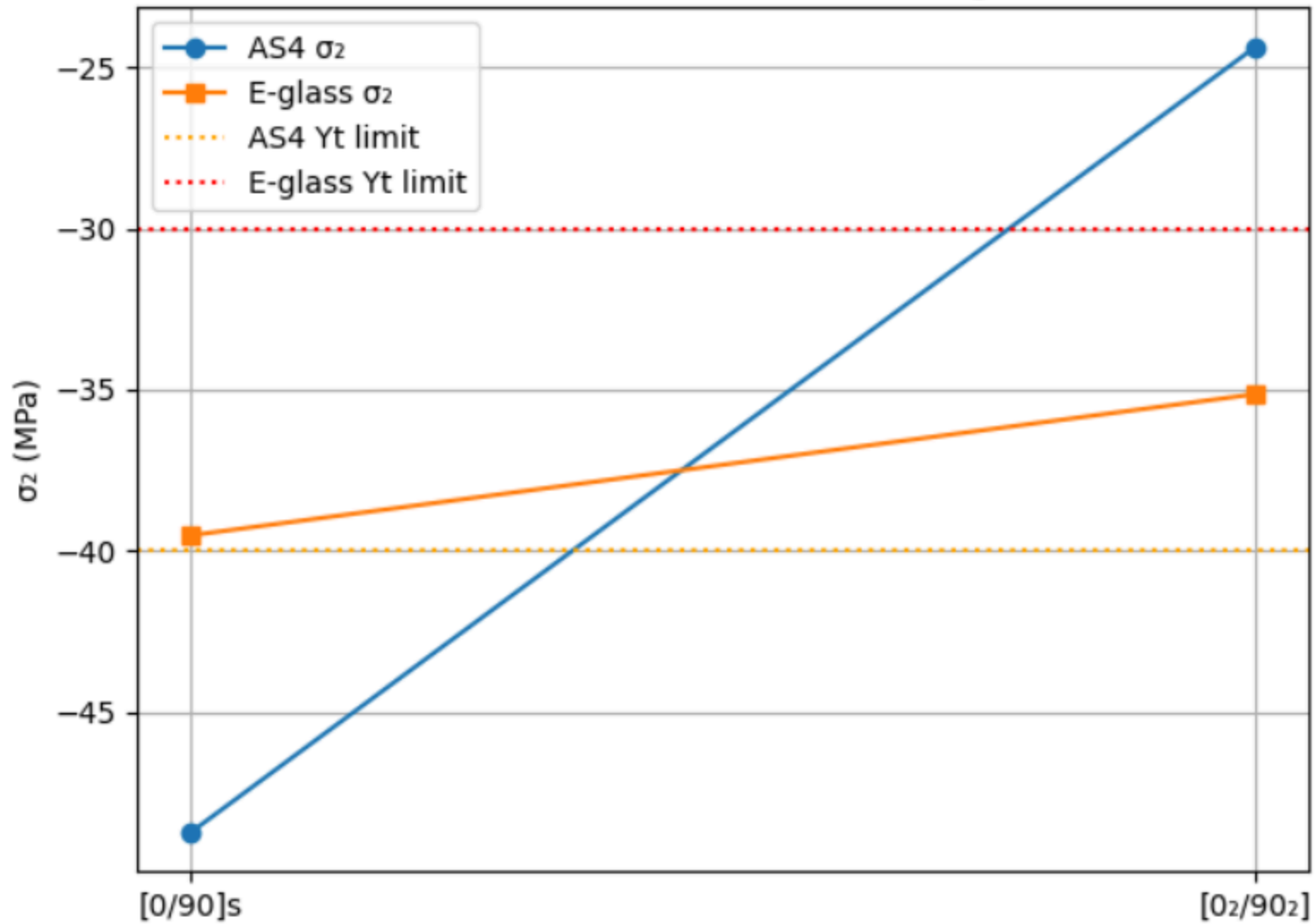
$[\theta/90]_s$: $E_x = 0.04$, $E_y = 0.04$, $G_{xy} = 0.24$ (GPa)
 $[\theta_2/90_2]_s$: $E_x = 0.04$, $E_y = 0.04$, $G_{xy} = 0.24$ (GPa)
 $[\theta_m/90_n/\pm 45_p]_s$: $E_x = 0.05$, $E_y = 0.05$, $G_{xy} = 0.16$ (GPa)



Part Two:



Thermal Residual Stress σ_2 with Strength Limits



Conclusion:

Elastic response is highly directional; AS4/3501-6 provides higher stiffness in 0° and superior resistance to thermal stress due to low α_1 . E-glass/epoxy demonstrates lower, balanced stiffness. Residual stress analysis confirms that all laminate designs remain within safe limits, validating both materials for structural use. This project allowed us to quantitatively investigate how the fiber orientation and lamina stacking sequences influences the elastic behavior and thermal stresses of these composite laminate materials. Additionally, these questions furthered our understanding and critical thinking towards informed design analysis of composite materials.

Appendix:

Part 1. Code:

```
#1

import numpy as np

import matplotlib.pyplot as plt

# Define material properties

materials = {

    "AS4/3501-6": {"E1": 181, "E2": 10.3, "G12": 7.17, "v12": 0.28},

    "E-glass/epoxy": {"E1": 38.6, "E2": 8.27, "G12": 4.14, "v12": 0.26}

}


# Q matrix and transformation

def get_Q_matrix(E1, E2, G12, v12):

    v21 = (E2 / E1) * v12

    denom = 1 - v12 * v21

    Q11 = E1 / denom

    Q22 = E2 / denom

    Q12 = v12 * E2 / denom

    Q66 = G12

    return Q11, Q22, Q12, Q66


def transform_Q(Q11, Q22, Q12, Q66, theta_deg):

    m = np.cos(np.deg2rad(theta_deg))
```

```
n = np.sin(np.deg2rad(theta_deg))
```

```
Q11_bar = Q11 * m**4 + 2*(Q12 + 2*Q66)*m**2*n**2 + Q22 * n**4
```

```
Q22_bar = Q11 * n**4 + 2*(Q12 + 2*Q66)*m**2*n**2 + Q22 * m**4
```

```
Q12_bar = (Q11 + Q22 - 4*Q66)*m**2*n**2 + Q12*(m**4 + n**4)
```

```
Q66_bar = (Q11 + Q22 - 2*Q12 - 2*Q66)*m**2*n**2 + Q66*(m**2 - n**2)**2
```

```
return Q11_bar, Q22_bar, Q12_bar, Q66_bar
```

```
# -----
```

```
# Compute effective moduli for lamina
```

```
# -----
```

```
def compute_effective_moduli(material_name, angles):
```

```
    props = materials[material_name]
```

```
    Q11, Q22, Q12, Q66 = get_Q_matrix(props["E1"], props["E2"], props["G12"], props["v12"])
```

```
    results = []
```

```
    for theta in angles:
```

```
        Q11_bar, Q22_bar, Q12_bar, Q66_bar = transform_Q(Q11, Q22, Q12, Q66, theta)
```

```
        E2_eff = Q22_bar - (Q12_bar**2 / Q11_bar)
```

```
        G12_eff = Q66_bar
```

```
        nu12_eff = Q12_bar / Q11_bar
```

```
        results.append([theta, E2_eff / props["E1"], G12_eff / props["G12"], nu12_eff])
```

```
    return np.array(results)
```

```

# -----

# Compute A matrix and extract laminate moduli

# -----

def compute_A_matrix(E1, E2, G12, v12, angles):

    Q11, Q22, Q12, Q66 = get_Q_matrix(E1, E2, G12, v12)

    A = np.zeros((3, 3))

    for theta in angles:

        Q11_bar, Q22_bar, Q12_bar, Q66_bar = transform_Q(Q11, Q22, Q12, Q66, theta)

        Q_bar = np.array([

            [Q11_bar, Q12_bar, 0],

            [Q12_bar, Q22_bar, 0],

            [0, 0, Q66_bar]

        ])

        A += Q_bar

    return A


def extract_laminate_moduli(A, total_thickness):

    A11, A22, A66 = A[0, 0], A[1, 1], A[2, 2]

    E_x = 1 / (A11 / total_thickness)

    E_y = 1 / (A22 / total_thickness)

    G_xy = 1 / (A66 / total_thickness)

    return E_x, E_y, G_xy

```

```

# -----

# Analyze laminate configurations

# -----

def analyze_laminate(material_name, angle_list, label):

    props = materials[material_name]

    A = compute_A_matrix(props["E1"], props["E2"], props["G12"], props["v12"], angle_list)

    Ex, Ey, Gxy = extract_laminate_moduli(A, total_thickness=len(angle_list))

    print(f'{label}: E_x = {Ex:.2f}, E_y = {Ey:.2f}, G_xy = {Gxy:.2f} (GPa)')


# -----

# Main execution

# -----

theta_range = np.arange(0, 91, 10)

qi_angles = [0, 90, 45, -45, -45, 45, 90, 0]

laminates = {

    "[0/90]s": [0, 90, 90, 0],

    "[02/902]": [0, 0, 90, 90],

    "[0m/90n/±45p]s": qi_angles

}


for name in materials:

    print(f'\n==== {name} ====')

```

```
# 1. Single-lamina transformed properties
```

```
result = compute_effective_moduli(name, theta_range)
```

```
print("Angle (deg) |  $E_2'/E_1$  |  $G_{12}'/G_{12}$  |  $\nu_{12}$ ")
```

```
for row in result:
```

```
    print(f"{int(row[0]):>11} | {row[1]:>6.3f} | {row[2]:>8.3f} | {row[3]:>6.3f}")
```

```
# Plot single lamina properties
```

```
plt.figure()
```

```
plt.plot(result[:, 0], result[:, 1], label='E2' / E1')  

```

```
plt.plot(result[:, 0], result[:, 2], label='G12' / G12')  

```

```
plt.plot(result[:, 0], result[:, 3], label='ν12')  

```

```
plt.xlabel("Fiber Orientation θ (degrees)")
```

```
plt.ylabel("Normalized Property")
```

```
plt.title(f'{name} - Normalized Lamina Moduli vs Fiber Angle')
```

```
plt.grid(True)
```

```
plt.legend()
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# 2. Analyze fixed laminate configurations
```

```
for label, angles in laminates.items():
```

```
    analyze_laminate(name, angles, label)
```



```

# 3. Analyze angle-ply laminate [ $\pm\theta$ ]s

angle_ply_thetas = np.arange(0, 91, 10)

Ex_list, Ey_list, Gxy_list = [], [], []

for theta in angle_ply_thetas:

    ply = [theta, -theta, -theta, theta]

    props = materials[name]

    A = compute_A_matrix(props["E1"], props["E2"], props["G12"], props["v12"], ply)

    Ex, Ey, Gxy = extract_laminate_moduli(A, total_thickness=len(ply))

    Ex_list.append(Ex)

    Ey_list.append(Ey)

    Gxy_list.append(Gxy)


# Plot angle-ply laminate results

plt.figure()

plt.plot(angle_ply_thetas, Ex_list, label="Ex (GPa)")

plt.plot(angle_ply_thetas, Ey_list, label="Ey (GPa)")

plt.plot(angle_ply_thetas, Gxy_list, label="Gxy (GPa)")

plt.xlabel("Fiber Angle  $\theta$  (degrees)")

plt.ylabel("Laminate Property")

plt.title(f"{name} - [ $\pm\theta$ ]s Laminate Properties vs  $\theta$ ")

plt.grid(True)

plt.legend()

plt.tight_layout()

```

```
plt.show()
```

Part 2 Code:

#2

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
# -----
```

```
# Material and strength properties
```

```
# -----
```

```
materials = {
```

```
    "AS4/3501-6": {
```

```
        "E1": 181e3, "E2": 10.3e3, "G12": 7.17e3, "v12": 0.28,
```

```
        "alpha1": 0.0e-6, "alpha2": 30e-6,
```

```
        "Xt": 1500, "Yt": 40 # MPa
```

```
    },
```

```
    "E-glass/epoxy": {
```

```
        "E1": 38.6e3, "E2": 8.27e3, "G12": 4.14e3, "v12": 0.26,
```

```
        "alpha1": 5e-6, "alpha2": 30e-6,
```

```
        "Xt": 1000, "Yt": 30 # MPa
```

```
    }
```

```
}
```

```
# -----
```

```

# Thermal stress calculation

# -----

def thermal_stress(mat, delta_T, layup):

    E1, E2, v12 = mat["E1"], mat["E2"], mat["v12"]

    alpha1, alpha2 = mat["alpha1"], mat["alpha2"]

    v21 = (E2 / E1) * v12

    if layup == "[0/90]s":

        sigma1 = -E1 * alpha1 * delta_T / (1 - v12 * v21)

        sigma2 = -E2 * alpha2 * delta_T / (1 - v12 * v21)

    elif layup == "[02/902]":

        # All layers same thickness, 2:2 ply ratio

        sigma1 = -((0.5)*E1*alpha1 + (0.5)*E2*alpha2) * delta_T / (1 - v12 * v21)

        sigma2 = -((0.5)*E2*alpha2 + (0.5)*E1*alpha1) * delta_T / (1 - v12 * v21)

    else:

        raise ValueError("Invalid layup")

    return round(sigma1, 2), round(sigma2, 2)

# -----

# Parameters

# -----

delta_T = 177 - 20 # °C

```

```

layups = ["[0/90]s", "[02/902]"
results = []

# Compute stresses
for mat_name, mat in materials.items():
    for layup in layups:
         $\sigma_1$ ,  $\sigma_2$  = thermal_stress(mat, delta_T, layup)
        results.append({
            "Material": mat_name,
            "Layup": layup,
            " $\sigma_1$  (MPa)":  $\sigma_1$ ,
            " $\sigma_2$  (MPa)":  $\sigma_2$ ,
            "Xt": mat["Xt"],
            "Yt": mat["Yt"]
        })

# -----
# Convert to DataFrame
# -----
df = pd.DataFrame(results)

# -----
# Plot with strength reference

```

```
# -----
```

```
x = np.arange(len(layups))
```

```
width = 0.35
```

```
# Plot  $\sigma_1$ 
```

```
plt.figure()
```

```
plt.plot(x, df[df.Material=="AS4/3501-6"][" $\sigma_1$  (MPa)"], marker='o', label="AS4  $\sigma_1$ ")
```

```
plt.plot(x, df[df.Material=="E-glass/epoxy"][" $\sigma_1$  (MPa)"], marker='s', label="E-glass  $\sigma_1$ ")
```

```
plt.axhline(-materials["AS4/3501-6"]["Xt"], color='orange', linestyle=':', label="AS4 Xt limit")
```

```
plt.axhline(-materials["E-glass/epoxy"]["Xt"], color='red', linestyle=':', label="E-glass Xt limit")
```

```
plt.xticks(x, layups)
```

```
plt.ylabel(" $\sigma_1$  (MPa)")
```

```
plt.title("Thermal Residual Stress  $\sigma_1$  with Strength Limits")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Plot  $\sigma_2$ 
```

```
plt.figure()
```

```
plt.plot(x, df[df.Material=="AS4/3501-6"][" $\sigma_2$  (MPa)"], marker='o', label="AS4  $\sigma_2$ ")
```

```
plt.plot(x, df[df.Material=="E-glass/epoxy"][" $\sigma_2$  (MPa)"], marker='s', label="E-glass  $\sigma_2$ ")
```

```
plt.axhline(-materials["AS4/3501-6"]["Yt"], color='orange', linestyle=':', label="AS4 Yt limit")
```

```
plt.axhline(-materials["E-glass/epoxy"]["Yt"], color='red', linestyle=':', label="E-glass Yt limit")

plt.xticks(x, layups)

plt.ylabel(" $\sigma_2$  (MPa)")

plt.title("Thermal Residual Stress  $\sigma_2$  with Strength Limits")

plt.legend()

plt.grid(True)

plt.tight_layout()

plt.show()
```